

Compras-Backend — Especificaciones

Versión 2026-09-23 — Refactorizar el sistema de *ventas de tienda* a un sistema de *tienda de comida* con venta local (POS) + venta online (storefront).

Backend

Angular (Local)

Next.js (Cliente)

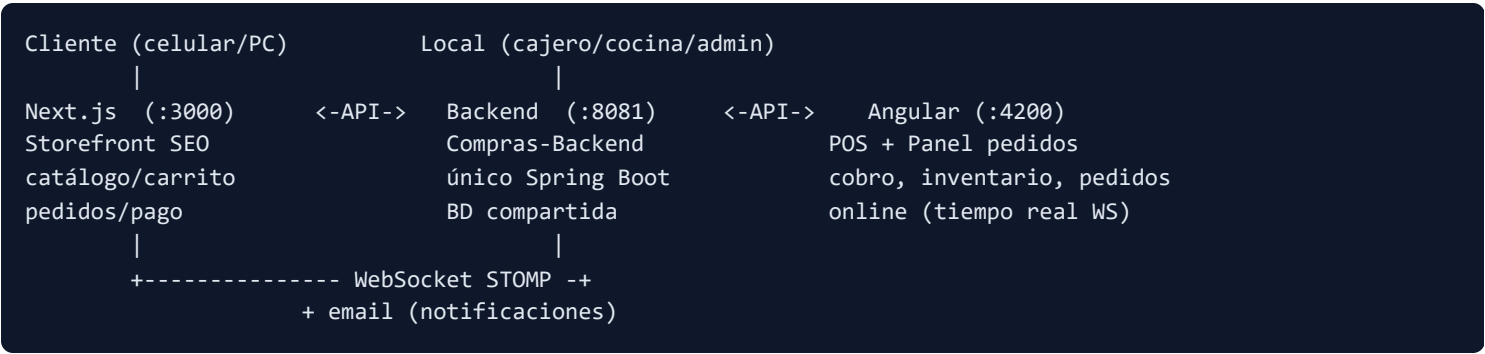
WebSocket

Stripe

0. Estado actual

Sistema	Ruta	Estado
Compras-Backend	Compras-Backend	Spring Boot 4 completo, paquete com.erikjarquin.ventas, 12 controllers / 57 endpoints / 130 tests. POS-focused (ventas, pagos terminal, caja, reportes).
Compras-Frontend-Local	Compras-Frontend-Local	Angular 21 standalone + SSR completo. Features en español: cobro (POS), productos, categorias, compras (3 tabs), reportes, salehistory, usuarios, roles, deactivated-*, sidebar data-driven con permisos. Sin módulos clientes/pedidos/caja.
Compras-Frontend-Cliente	Compras-Frontend-Cliente	Solo .git (vacío) → el Next.js nace aquí.

1. Arquitectura objetivo



- **Un solo backend, una sola BD.** Venta local y Pedido online son entidades **separadas** con ciclos de vida distintos.
- Dos dominios independientes: **POS/gestión** y **tienda online**. Se separan por prefijo de ruta y por tipo de usuario (EMPLEADO vs CLIENTE).

¿Por qué dos frontends separados y no uno solo? Porque los usuarios son distintos y necesitan experiencias distintas:

Aspecto	Frontend Local (Angular)	Frontend Cliente (Next.js)
Usuario	Cajero, admin, empleado	Cliente final
Dispositivo	PC / tablet en el local	Celular, PC en casa

Objetivo	Cobrar rápido, gestionar	Ver menú, comprar, seguir pedido
SEO	No importa	Sí importa (que te encuentren)
Autenticación	Login de empleado	Login de cliente (o invitado)
Diseño	Denso, funcional	Atractivo, marketing
Velocidad de uso	Muy rápida (muchos clics)	Fluida pero no crítica

2. Backend — qué se refactoriza y qué se agrega

2.1 Refactor base (naming ventas → compras)

- Paquete `com.erikjarquin.ventas` → `com.erikjarquin.compras`; `VentasApplication` → `ComprasApplication`; `pom.xml` `artifactId` `compras`; `spring.application.name`; DB `compras_db` (BD nueva limpia, recomendado) o migrar la existente.
- **Separar rutas por prefijo**: lo actual pasa a `/api/local/**` (POS/gestión) y lo nuevo queda en `/api/tienda/**` (cliente). Seguridad: `/api/tienda/productos/**` público de solo lectura; el resto autenticado.
- Actualizar README, AGENTS.md, Dockerfile y javadocs.

2.2 Modelo de datos nuevo

```

ClienteEntity (id, nombre, email UNIQUE, password BCrypt, telefono, direccion,
               fechaRegistro)                <- rol CLIENTE, tabla propia
PedidoEntity (id, numero, cliente, fecha, estado, tipoEntrega[RECOGER/DOMICILIO],
              metodoPago[EFFECTIVO/TRANSFERENCIA/STRIPE], estadoPago, total,
              direccion/envio, notas, canal=ONLINE)
DetallePedidoEntity (id, pedido, platillo, cantidad, precioUnitario, subtotal)
InsumoEntity (id, nombre, unidad[kg/L/pza], stock, costoUnitario, stockMinimo)
PlatilloEntity <- forma final de Producto (platillo + receta)
RecetaDetalleEntity (id, platillo, insumo, cantidadNecesaria)

```

- **Crítica del modelo**: Platillo ya no acumula stock — su disponibilidad se **deriva de la receta**: si los insumos alcanzan para preparar N, el platillo está disponible. Quedan 2 opciones (ver Decisiones, apartado 6).
- **Producto** → **Platillo**: se quita barcode/sku (o quedan opcionales, null), se agrega descripción ("hamburguesa de pollo con BBQ"), imagen se mantiene. El formulario del frontend se reutiliza casi igual.

2.3 Módulos nuevos

1. **Insumos** (CRUD + stock). Cada venta/cancelación de pedido **descuenta/regresa insumos** según receta (o simplemente reporta "insumos insuficientes"). Endpoints `/api/local/insumos`.
2. **Clientes** + auth de cliente (POST `/api/tienda/auth/registro`, `.../login`, `.../me`). JWT con claim `tipo=CLIENTE|EMPLEADO` para que `JwtFilter` sepa a qué tabla buscar. No toca el sistema de empleados.
3. **Pedidos**:

- `/api/tienda/pedidos` — crear (valida stock/insumos), mis pedidos, seguimiento, cancelar (solo en PENDIENTE).
 - `/api/local/pedidos` — listar por estado, PATCH `/id/estado` (CAJERO/COCINA acepta → prepara → listo → entregado), ver detalle.
 - Estados: PENDIENTE → CONFIRMADO → EN_PREPARACION → LISTO → ENTREGADO/RECOGIDO + CANCELADO/RECHAZADO.
4. **Pagos online:** pasarela (Stripe test primero). Modelo: `Pedido.estadoPago` + tabla `TransaccionOnline` (opcional). El local nunca ve datos de tarjeta.
5. **WebSocket (STOMP):** al crear pedido, backend hace `convertAndSend("/topic/pedidos", dtoPedido)`. Angular suscribe. **Este es el puente Next.js → Angular.**
6. **Notificaciones (email):** ya existe `spring-boot-starter-mail`. Email al cliente en cambios de estado (listo para enviar con `FRONTEND_URL` del storefront).

2.4 Endpoints resultantes (vista rápida)

Prefijo	Endpoints	Notas
<code>/api/tienda/productos</code>	GET catálogo, GET <code>/categorias</code> , GET <code>/id</code>	público
<code>/api/tienda/auth</code>	registro, login, me	cliente
<code>/api/tienda/pedidos</code>	POST, GET <code>/mios</code> , GET <code>/id</code> , PATCH <code>/id/cancelar</code>	
<code>/api/local/pedidos</code>	GET <code>?estado=</code> , GET <code>/id</code> , PATCH <code>/id/estado</code>	panel empleado
<code>/api/local/insumos</code>	CRUD + bajo stock	
<code>/api/local/products</code>	CRUD platillos (barcode/sku opcional)	renombrado desde <code>/products</code>
<code>/ws/**</code> (STOMP)	<code>/topic/pedidos</code>	suscripción Angular

2.5 Permisos y roles nuevos

- Permisos (34 → ~40): `VER_CLIENTES`, `VER_PEDIDOS`, `ACCIONAR_PEDIDOS` (aceptar/preparar/entregar), `VER_INSUMOS`, `CREAR_INSUMOS`, `EDITAR_INSUMOS`, `ELIMINAR_INSUMOS`.
- Roles: se mantienen ADMIN/CAJERO/ALMACENISTA. **CAJERO** recibe `VER_PEDIDOS` + `ACCIONAR_PEDIDOS` (panel de cocina); se agrega rol **CLIENTE** (solo permisos de tienda, no entra al POS).

2.6 Reportes

- Unificar canales: agregar columna `canal` (LOCAL/ONLINE) para que las queries existentes (trend, top-products, summary, margins) sumen ambos, con filtro opcional por canal. Venta local sigue siendo `SaleEntity`; lo online se agrega al reporte vía `Pedido pagado`.

3. Frontend Angular local — qué se refactoriza

1. **Ajuste de catálogo:** formulario de producto → platillo (quitar SKU/código de barras del alta y de la búsqueda del POS; agregar descripción). Categorías → reetiquetar a tipos de platillo (Hamburguesas, Pizzas, Bebidas...).
2. **Nuevo feature insumos/:** CRUD inventario de materia prima + alerta de stock bajo (se conecta al POS: valida que alcancen los insumos).
3. **Nuevo feature pedidos-online/:** panel — columnas NUEVOS / EN PREPARACIÓN / LISTOS / COMPLETADOS, botones Aceptar/Rechazar/Preparado/Entregar, conexión STOMP (@stomp/stompjs + SockJS) a /topic/pedidos.
4. **Servicios:** los actuales apuntan a /api/... → cambian a /api/local/.... Nuevos servicios: insumo-service, pedido-service, ws-service.
5. **Sidebar:** nuevos items Pedidos Online (VER_PEDIDOS), Insumos (VER_INSUMOS); reetiquetar Productos → Platillos.
6. Solo repaginar nombres visuales (ventas→compras/platillos); el POS /cobro se mantiene como núcleo.

4. Frontend Next.js cliente — qué se construye

- **Stack:** Next.js App Router + TypeScript + Tailwind. SSR para SEO: home, catálogo y detalle de platillo se renderizan en servidor consumiendo /api/tienda/productos.
- **Rutas:** / (menú), /categoria/[slug], /platillo/[id], /carrito, /checkout, /login, /registro, /pedidos (mis pedidos + seguimiento con polling suave).
- **Carrito:** Context + localStorage en el cliente (fase inicial); al hacer checkout se valida stock contra el backend. Persistir carrito en BD = opcional (fase 2).
- **Auth cliente:** consume /api/tienda/auth. Invitado puede armar carrito; al checkout requiere registro/login.
- **Checkout:** dirección/tipo de entrega (recoger/domicilio) + método de pago (STRIPE test → pasarela; transferencia → muestra cuenta/CBU + subir comprobante).
- **Seguimiento:** no necesita WebSocket; polling cada ~10s de /api/tienda/pedidos/{id}.

5. Hoja de ruta (paso a paso, con verificación en cada fase)

- **Fase 1 — Base backend:** renombrar ventas→compras (paquete/clases/artefacto/DB), dividir prefijos /api/local, actualizar README/AGENTS. → mvn test y Angular volviendo a apuntar a /api/local en verde.
- **Fase 2 — Platillos + Insumos + Recetas** (backend y luego UI Angular): modelo de insumos, receta, migración producto→platillo con descripción, endpoints. Verificación: CRUD de insumos en Angular + POS validando disponibilidad.
- **Fase 3 — Cliente + rol CLIENTE + auth:** entidad, JWT con tipo, endpoints /api/tienda/auth, CORS agrega http://localhost:3000.
- **Fase 4 — Pedidos + DetallePedido + estados + WS:** endpoints tienda/local, WebSocket STOMP, número de pedido. Verificación: crear pedido por curl → evento llega al clon Angular.
- **Fase 5 — Pago online (Stripe test):** checkout con tarjeta en modo test; estadoPago. Sin datos de tarjeta en el backend (token).

- **Fase 6 — Next.js (Compras-Frontend-Cliente):** scaffold → catálogo SSR → carrito → checkout (sin pago real) → login/registro cliente → mis pedidos. ng build/next build OK.
- **Fase 7 — Angular panel pedidos + insumos UI + sidebar:** pedidos en tiempo real (STOMP), botones de estados, alertas de insumos bajos, reetiquetado visual.
- **Fase 8 — Pulido:** emails de estado, reportes con canal online, permisos en pantalla de Roles, environment-prod para ambos frontends, cobertura de tests nuevos.

6. Decisiones a confirmar antes de empezar

1. **BD nueva limpia** (compras_db, sin datos) o migrar la ventas_db existente con datos de prueba.
2. **Stock de platillo:** (a) derivado de insumos por receta (más "chef", recomendado) o (b) platillo con stock aparte + insumos solo informativos.
3. **Pasarela:** Stripe (test) vs MercadoPago (más común en LatAm).
4. **Delivery real** desde el inicio o solo "recoger en tienda" primero.
5. **Lector de código de barras:** quitar del todo o dejarlo opcional (existen códigos en bebidas/empaques).